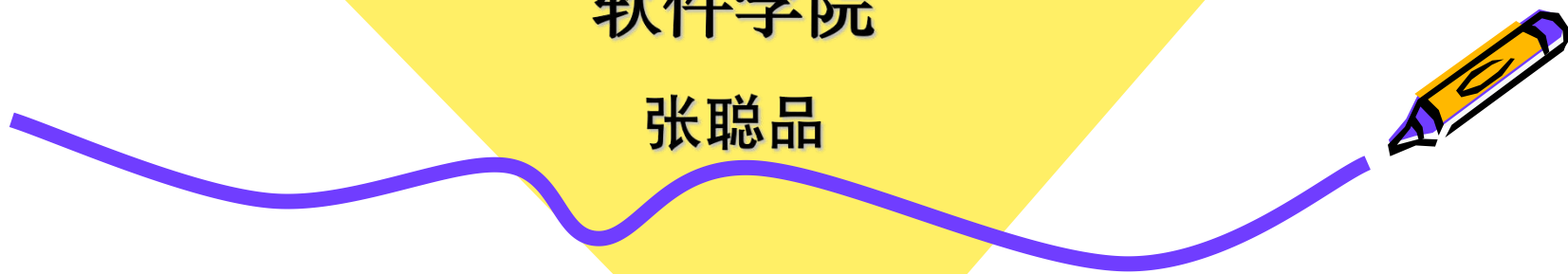




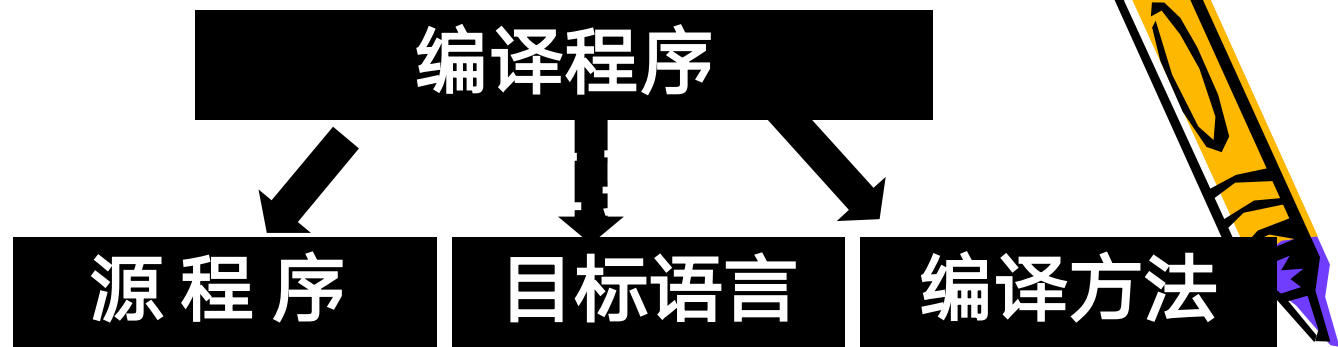
编译原理及实践教程

河南师范大学
软件学院

张聪品



回顾



1 手工构造
词法分析器

2 自动构造词
法分析器

词法分析

语法分析

语义分析与中

间代码生成

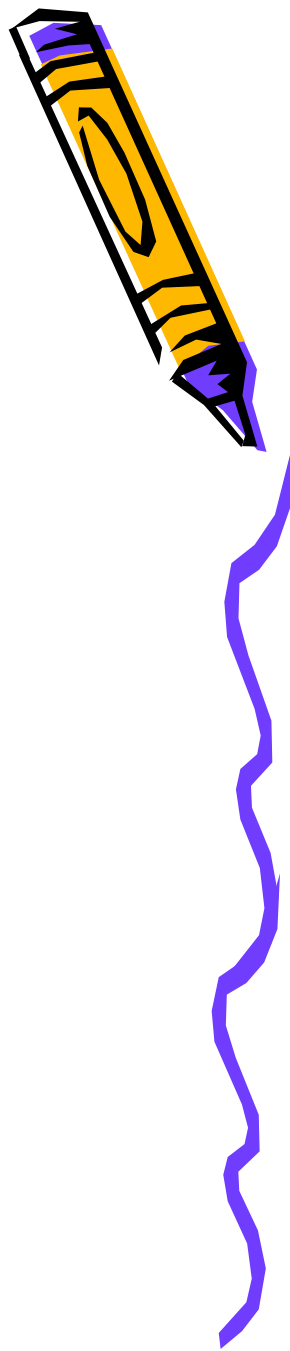
代码优化

目标代码生成



词法分析器的自动生成

- 正规式
- 有穷自动机

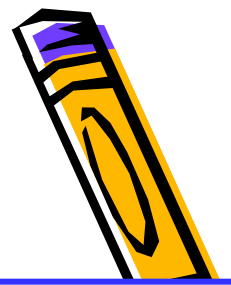


回顾

一个确定的有穷自动机 (DFA) M 是一个五元组: $M = (S, \Sigma, \delta, s_0, F)$, 其中:

1. S 是一个有穷集, 它的每个元素称为一个状态;
2. Σ 是一个有穷字母表, 它的每个元素称为一个输入符号, 所以也称 Σ 为输入符号表;
3. δ 是转换函数, 是在 $S \times \Sigma \rightarrow S$ 上的单值映射, $\delta(s, a) = s'$ ($s \in S, s' \in S$), 就意味着, 当前状态为 s , 输入符为 a 时, 将转换为下一个状态 s' , 我们把 s' 称作 s 的一个后继状态;
4. $s_0 \in S$ 是唯一的一个初态;
5. $F \subseteq S$ 是一个终态集(可空), 也称可接受状态或结束状态。

回顾



- ❖ 状态集合 I 的 ε -闭包 $\varepsilon\text{-closure}(I)$ ，是一状态集
- ❖ 任何状态 $q \in I$ ，则 $q \in \varepsilon\text{-closure}(I)$ ；
- ❖ 任何状态 $q \in I$ ，则 q 经任意条 ε 弧而能到达的状态 $q' \in \varepsilon\text{-closure}(I)$ 。

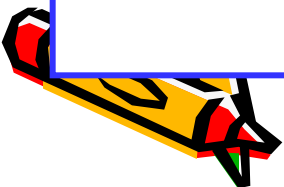
- ❖ 状态集合 I 的 a 弧转换， $Ia = \varepsilon\text{-closure}(J)$ ，其中 $J = \text{move}(I, a)$ ，即所有可从 I 中的某一状态经过一条 a 弧而到达的状态的全体。



回顾



1. $\Sigma = \{a_1 \cdots a_k\}$, 则初始时该表有 $k+1$ 列, 分别为 I 、 $Ia_1 \cdots Ia_k$, 首行首列为 $\varepsilon\text{-closure}(X)$, (X 为开始结点);
 2. 每行的值 $Ia_k = \varepsilon\text{-closure}(\text{move}(I, a))$, 其行数未知;
- ✧ 将新产生的 Ia_k 集合加入到 I 中作为新的一行, 并求该集合 I 的 $Ia_1 \cdots Ia_k$, 重复之, 直到状态不再增加; **初态** 就是首行首列的状态, **终态** 是含有原终态的集合。
- ✧ 将每个状态子集视为一个新的状态





4 DFA的化简

与某一NFA等价的DFA不一定唯一。

不同的DFA识别的正规集可能是相同的

每一个正规集都可以由一个状态数最少的DFA所识别，这个DFA是唯一的（因状态名不同的同构情况除外）。



DFA的最小化



- DFA的最小化就是寻求状态数最少的DFA，即：
 - 它没有多余状态；（消去）
 - 它的状态中没有两个是互相等价的。（合并）
- 多余状态是指：从开始状态出发，任何输入串也不能到达的那个状态；或者从这个状态没有通路到达终态。
- 状态S和T等价的条件
 - ◇ 一致性条件 —— 状态S和T必须同时为可接受状态或不可接受状态。
 - ◇ 蔓延性条件 —— 对于所有输入符号，状态S和状态T必须转换到等价的状态里。



DFA的最小化的方法——分割法

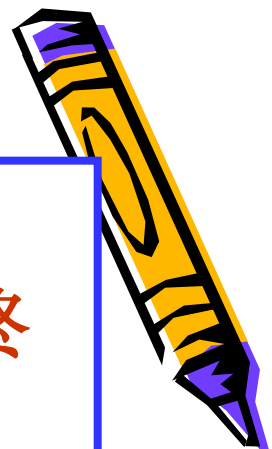
- 分割法的核心

- 把DFA的全部状态划分成一些互不相交的子集，使得任何不同的两子集的状态都是可区别的（不等价），而同一子集中的任何两个状态都是等价的。

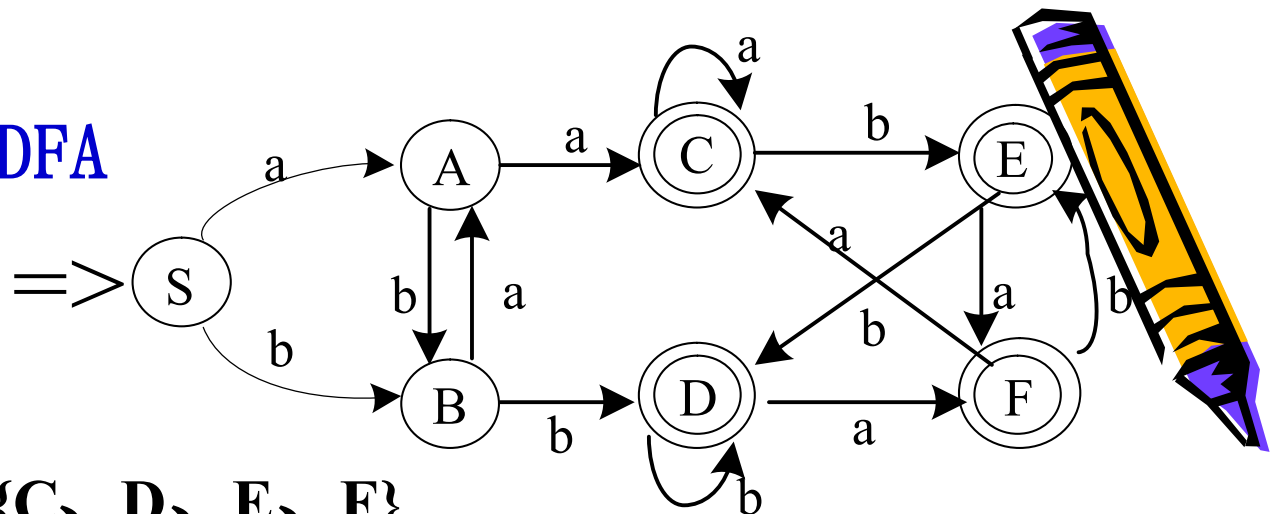


• 算法：

- 所有状态分成两个子集——**终态集**和**非终态集**；
- 运用判定状态**等价的原则**分别对两个子集的状态进行分析和划分，若发现某个状态与其它状态不等价，则将其作为一个新的状态子集，如果无法区分，则放在同一子集中；
- 从每个子集中选出一个状态做代表，即可构成简化的DFA；
- 含有原来初态的子集仍为**初态**，各终态的子集仍为**终态**。



例：化简下图的DFA



Π_0 : {S, A, B} {C, D, E, F}



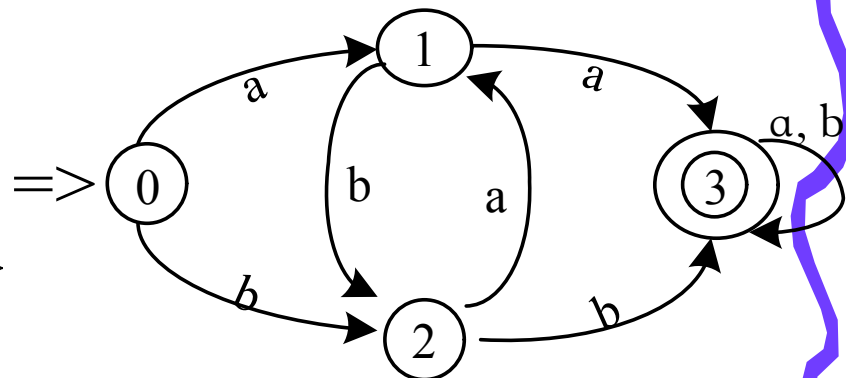
Π_1 : {A} {S, B} {C, D, E, F}



Π_2 : {A} {S} {B} {C, D, E, F}

选C为
代表
 $\downarrow$

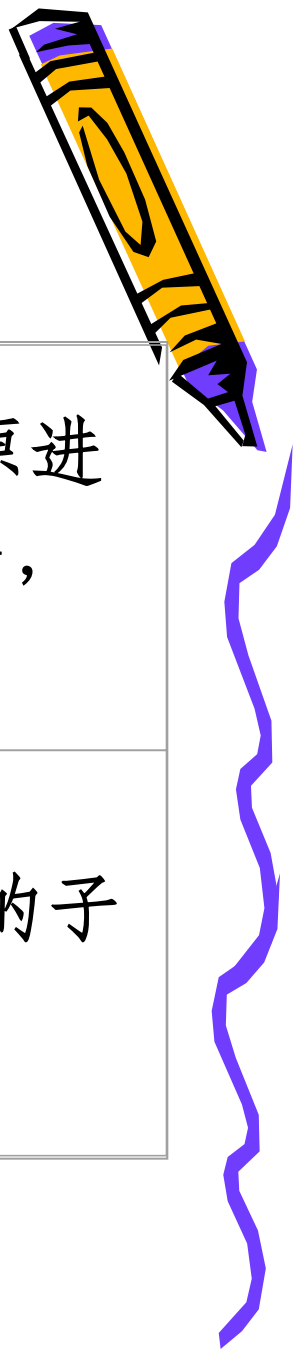
{A} {S} {B} {C}



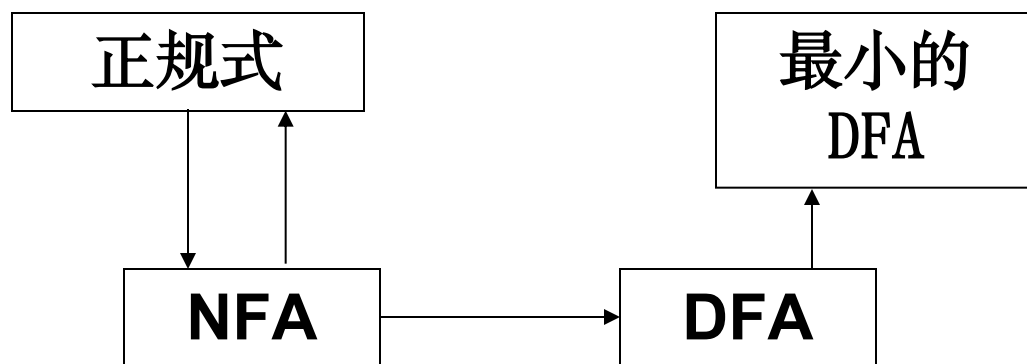
• 合并状态注意：

a、由于一个子集中，各状态等价，故只需将原进入该子集中各状态的弧都改为进入所选的状态，子集中各状态射出的弧均改为从该状态射出。

b、含有原来初态的子集仍为初态，含原终态的子集仍为终态



· 正规集的各种描述工具及其相互间的转换

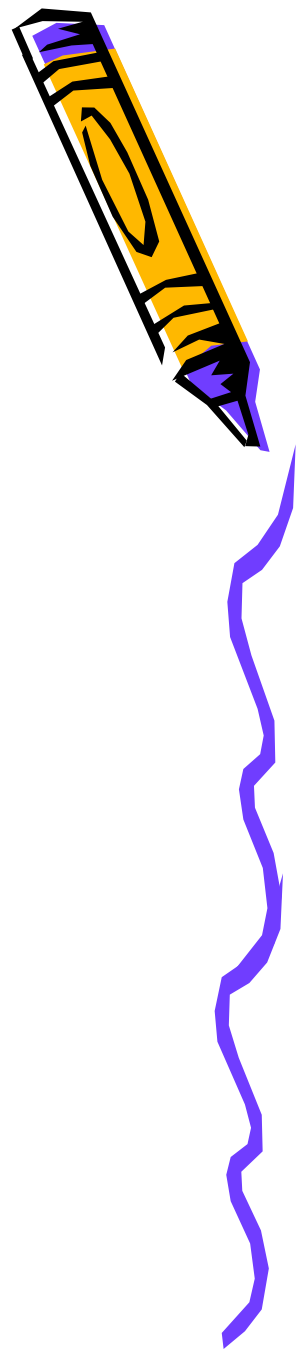


2.5.5 正规式与有限自动机的等价性

正规式和有穷自动机的等价性有以下两点说明：

※ 对于 Σ 上的 NFA M ，可以构造一个 Σ 上的正规式 R ，使得 $L(R) = L(M)$ 。

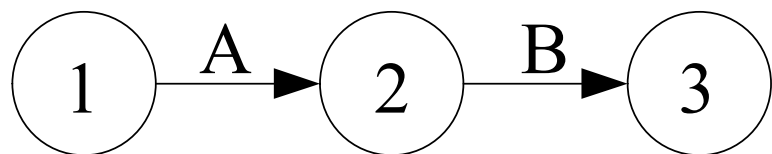
※ 对于 Σ 上的每个正规式 R ，可以构造一个 Σ 上的 NFA M ，使得 $L(M) = L(R)$ 。



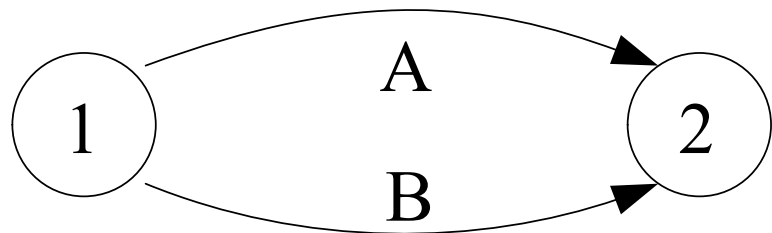
1 有穷自动机 $M \rightarrow$ 正规式 R

方法：

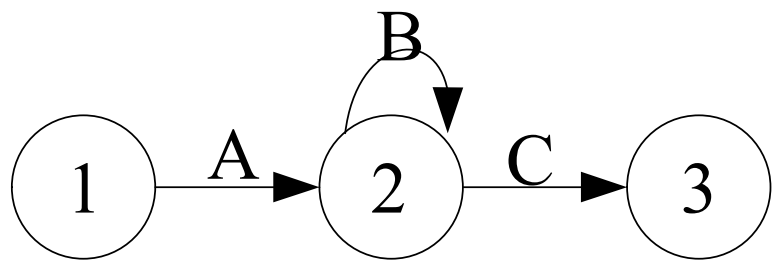
- 将状态转换图的概念拓广，每一条弧用一个正规式作标记。
- 增加结点 X, Y ，使之成为新的唯一的初态和终态。从 X 引 ϵ 弧到原初态结点，从原终态结点引 ϵ 弧到 Y 结点。
- 利用如下的规则消去新的状态图中的所有结点，直至只剩下 X, Y 结点。
- 最后得到从 X 到 Y 弧上的正规式。



代之以:



代之以:

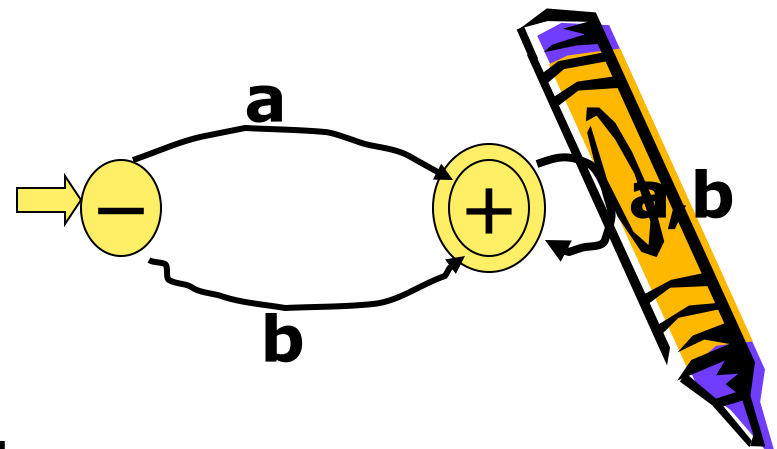


代之以:

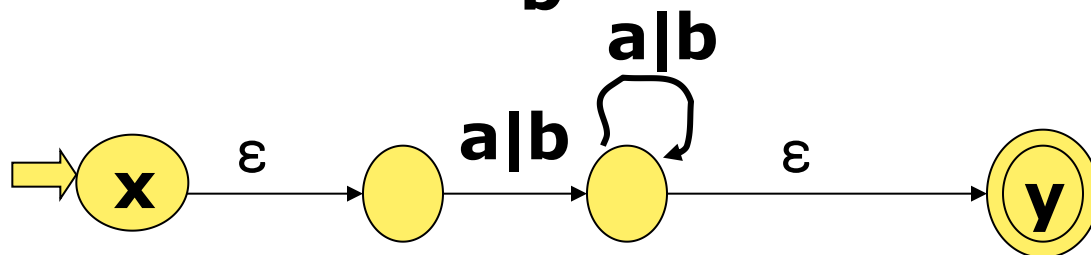
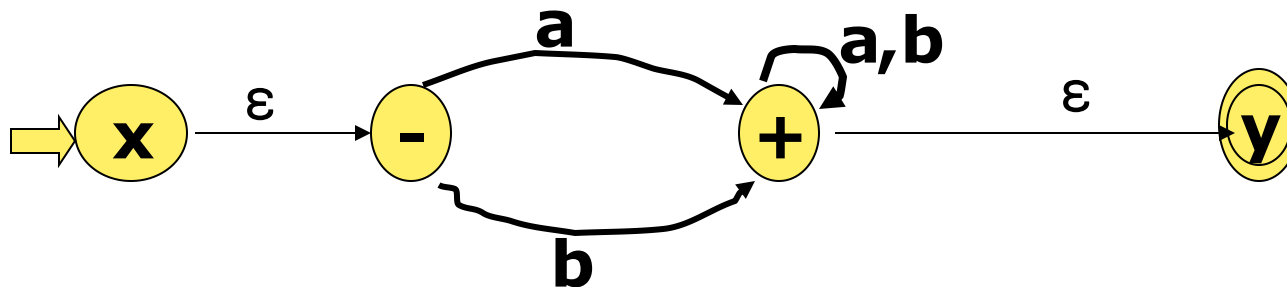


例：L(M) 如右图：

求正规式R，使 $L(R)=L(M)$ 。



解：



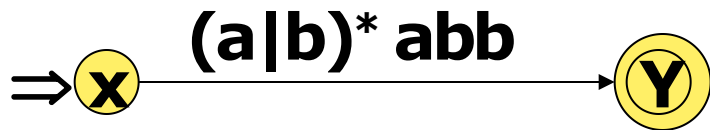
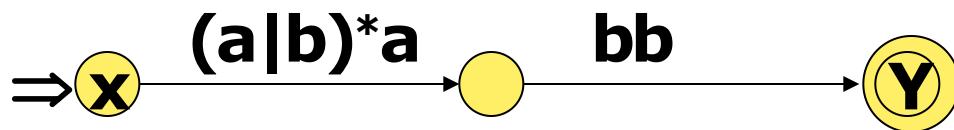
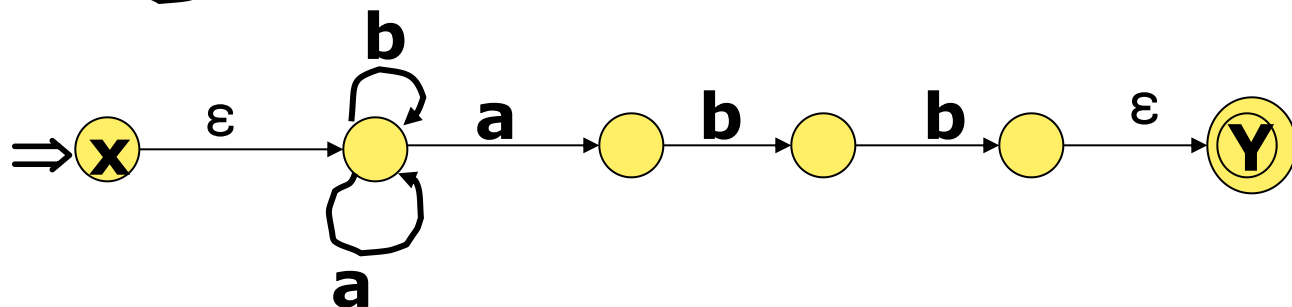
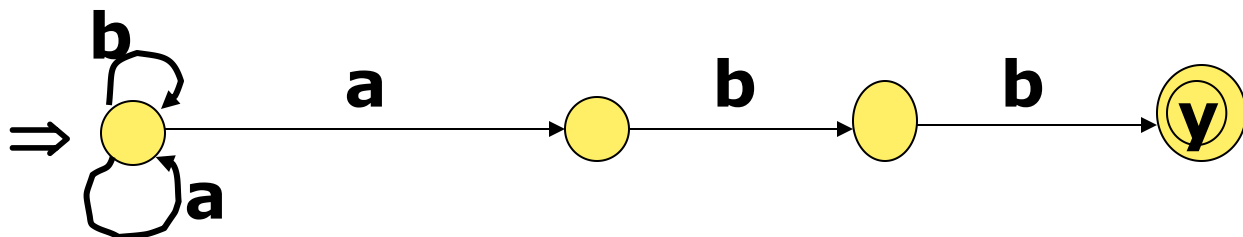
因此：

$$L(R) = (a|b)(a|b)^*$$

练习



求下述有穷自动机对应的正规式



$L(R) = (a|b)^*abb$



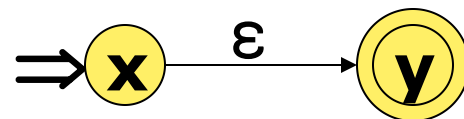
2 正规式 $R \rightarrow$ 有穷自动机NFA M

方法如下：

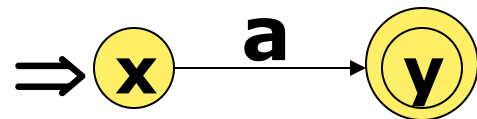
① 正规式 ϕ ，构造NFA为：



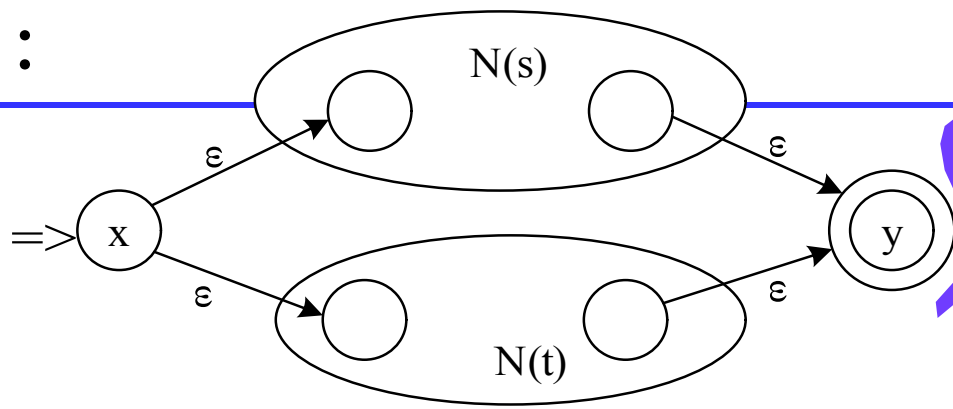
② 对应正规式 ε ，构造NFA为：



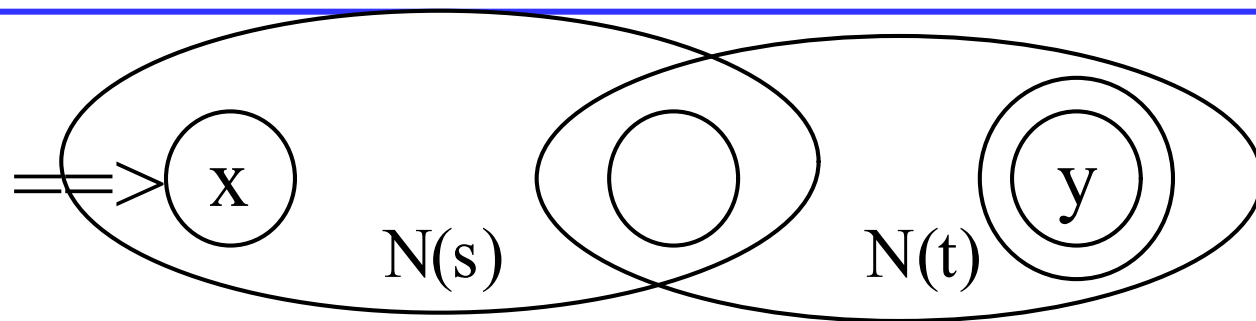
③ 对应正规式 a ，构造NFA为：



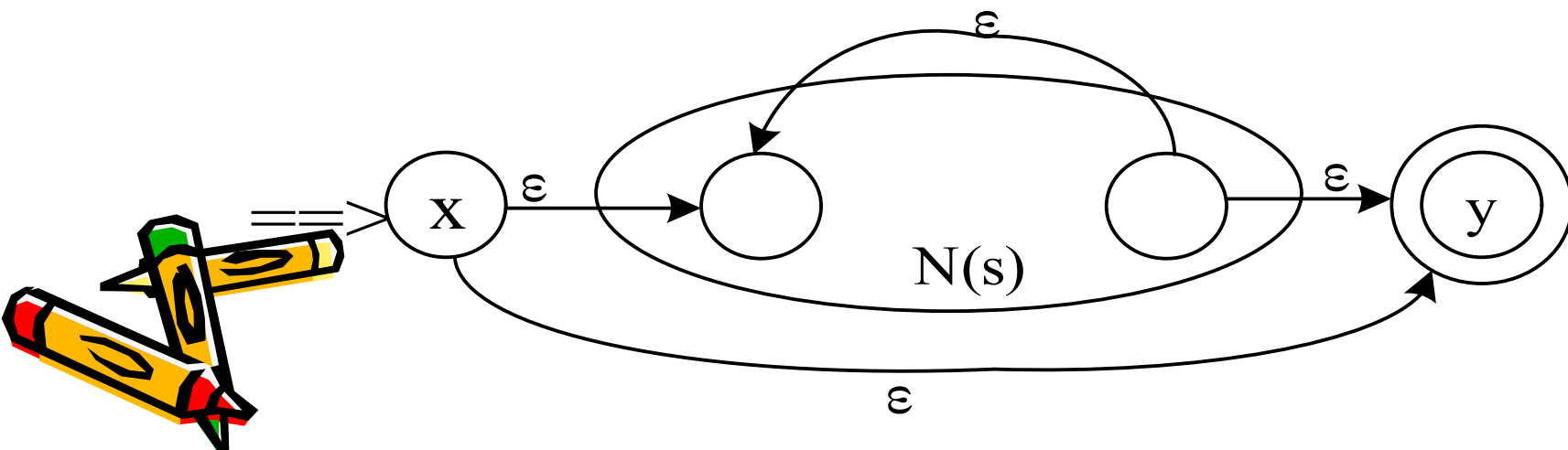
④ s, t 是正规式，相应NFA为 $N(s), N(t)$ ，则正规式 $R=s|t$ ，构造NFA(R) 为：



⑤ s, t 是正规式，相应NFA为 $N(s), N(t)$ ，则正规式 $R=st$ ，构造NFA (R) 为：

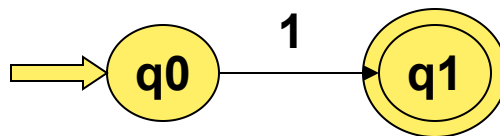


⑥ s 是正规式，相应NFA为 $N(s)$ ，则正规式 $R=s^*$ ，构造NFA (R) 为：

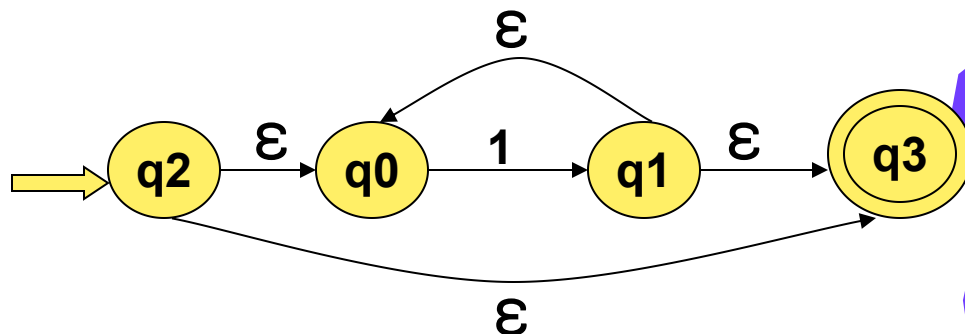


教材例2.17

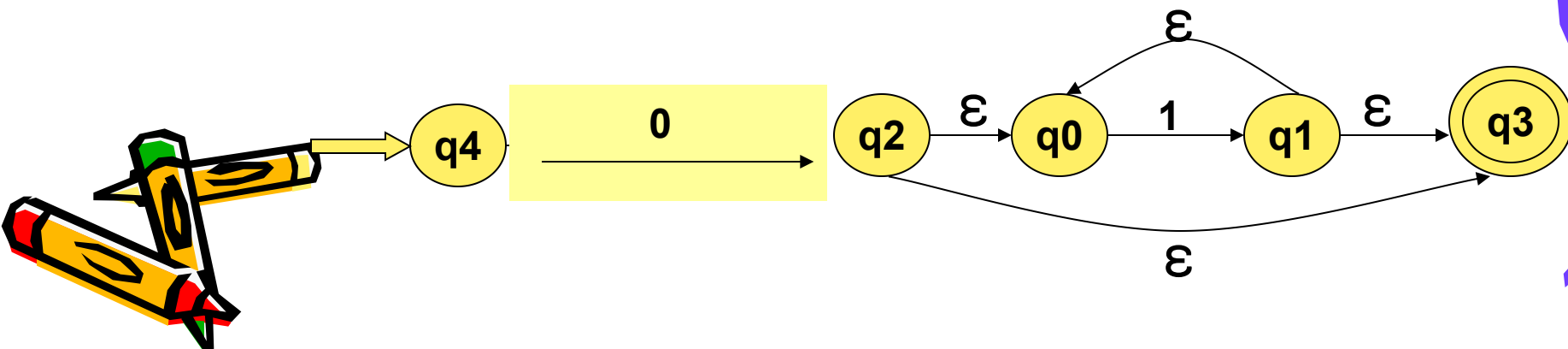
$R = 1$ 的有穷自动机:



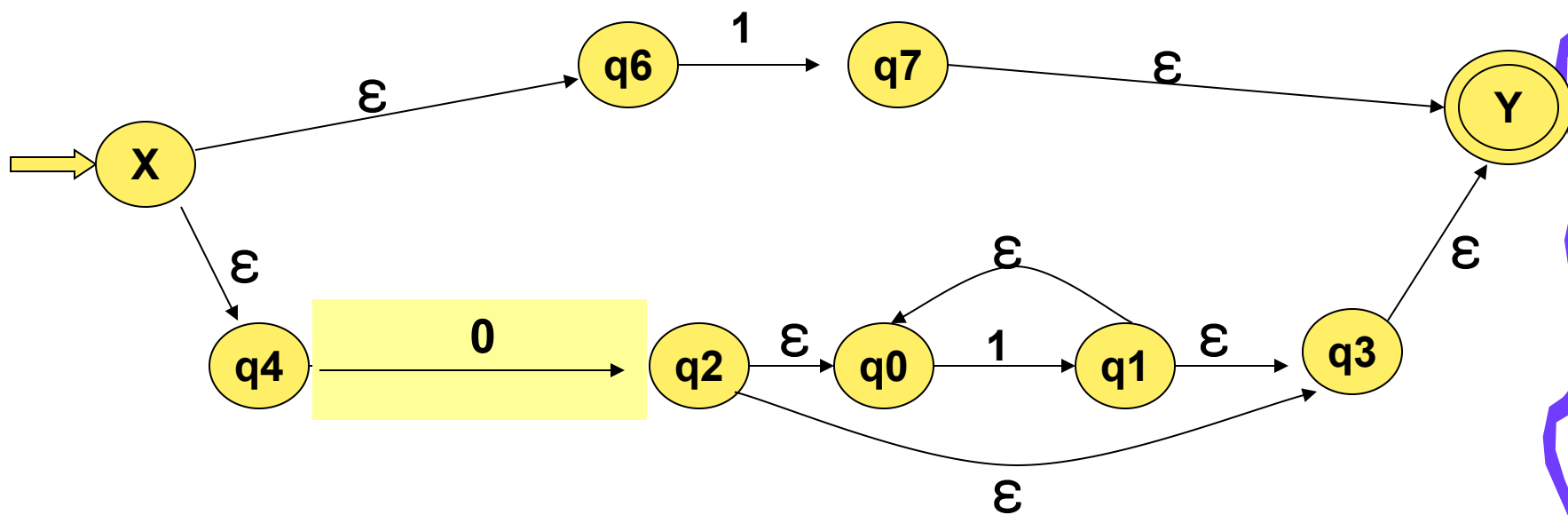
$R = 1^*$ 的有穷自动机:



$R = 01^*$ 的有穷自动机:

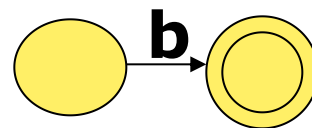
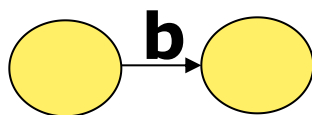
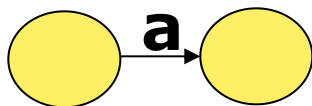
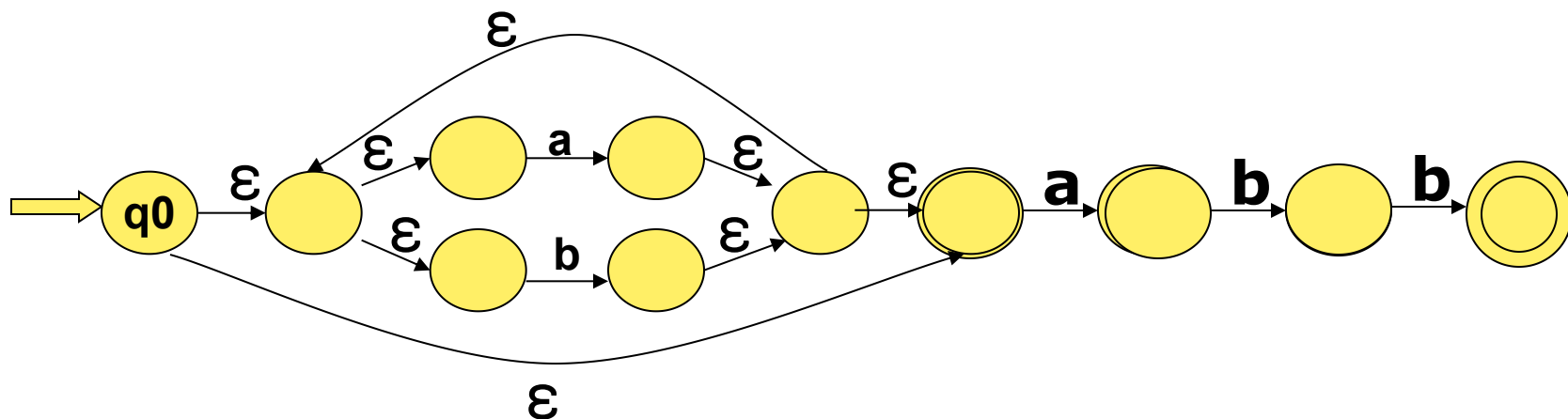
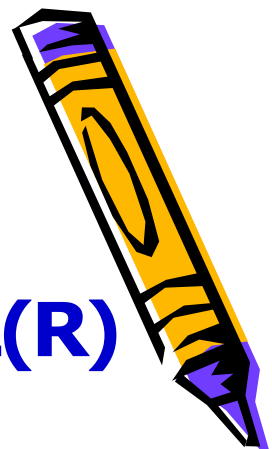


$R = 01^* | 1$ 的有穷自动机:



练习

正规式 $L(R) = (a|b)^*abb$, 构造NFA使 $L(N) = L(R)$



2.6 词法分析程序的自动生成工具

对有穷自动机和正规表达式进行了上述讨论之后，词法分析程序的自动构造方法基于有穷自动机和正规式的等价性。

1. 词法分析程序的自动构造程序，将其中的正规式转换为相应的NFA，再转换为与之等价的DFA，对DFA化简，最后产生用该DFA驱动的C语言词法分析函数。
2. 词法分析程序的自动构造程序LEX。LEX是一个基于正规式的描述构造词法分析程序的工具，也称LEX编译器。它的输入是LEX源程序（正规式的说明和对应的动作）输出是词法分析的C语言程序。



2.6.1 Lex概述

Lex是LEXical compiler的缩写，是Unix环境下非常著名的工具，主要功能是生成一个词法分析器(scanner)的C源码，描述规则采用正则表达式(regular expression)。

它的输入是用Lex语言编写的源程序，在lex源程序中，要将基于正则表达式的模式说明与词法分析器要完成的动作组织在一起。输出是词法分析的C语言程序。



声明部分

Lex文件结构分为三个部分：

declarations

(声明) %%

translation rules

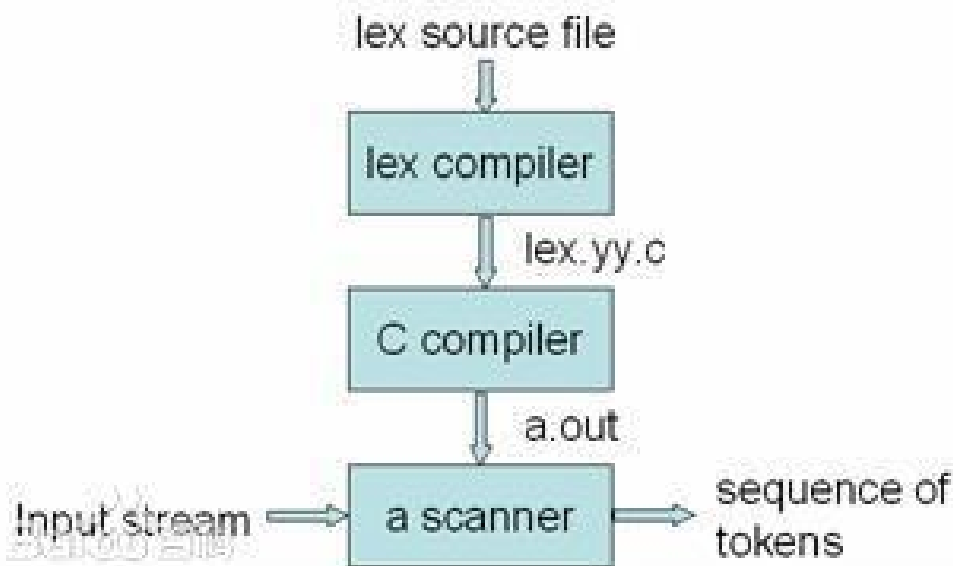
(翻译规则) %%

auxiliary

procedures

(其它函数)

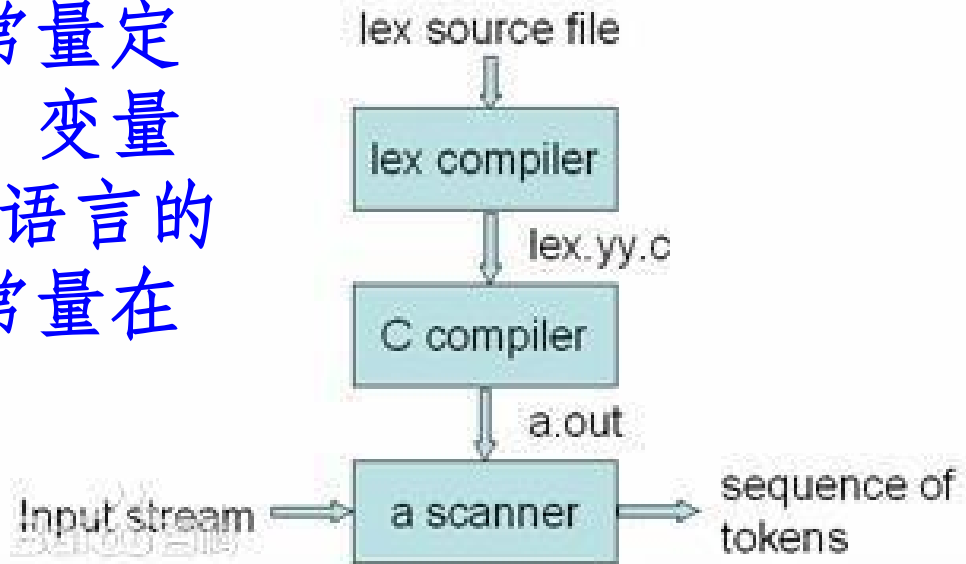
(辅助程序)



声明段包括变量的声明、符号常量的声明和正则表达式声明。



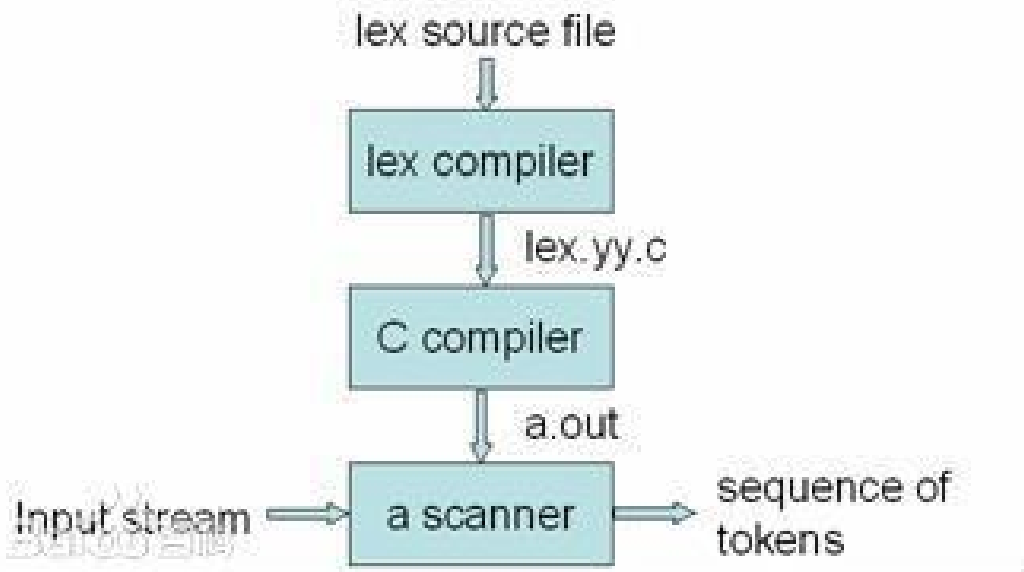
声明包括变量声明、常量定义和正则表达式定义。变量声明、常量定义遵循C语言的定义，定义的变量和常量在后续的C程序中使用。



希望出现在目标C源码中的代码，用
%{...%}扩在一起。如：

```
%{  
#include <stdio.h>  
#include "y.tab.h"  
typedef char * YYSTYPE;  
char * yylval;  
% }
```



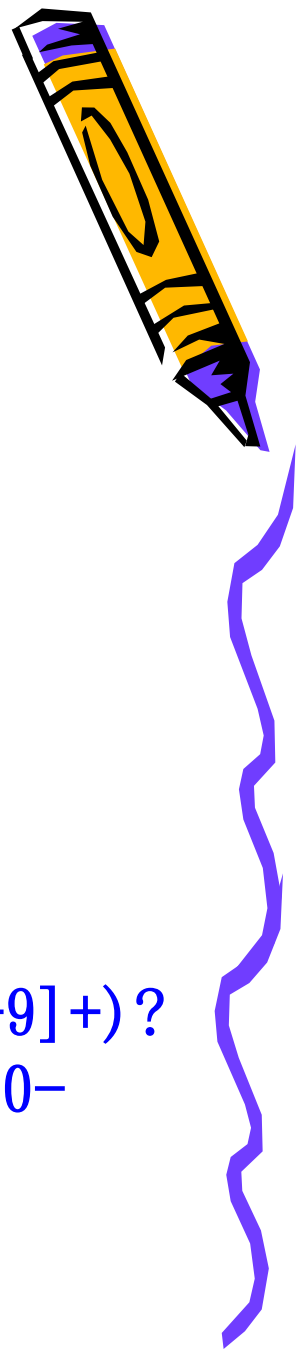
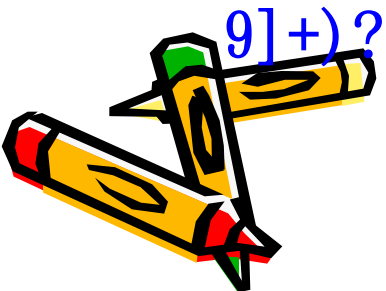


正则表达式的声明:

正则表达式的名字 → 正则表达式

十进制浮动数的正规式定义:

Float number → (“+” | ” -”)? [0-9]+ (“.” [0-9]+)? (E (“+” | ” -”)? [0-9]+)?



翻译规则：

P1 { 动作1 }

P2 { 动作2 }

 .

 .

 .

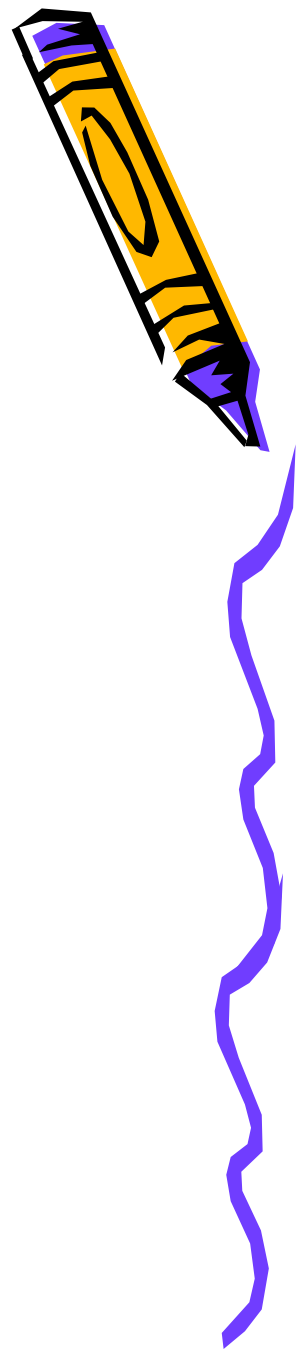
Pi { 动作i }

 .

 .

 .

Pn { 动作n }



本章小结

- 本章要求

- 1. 词法分析器的作用和接口，用高级语言编写词法分析器。
- 2. 掌握正规式和有穷自动机的相关概念，它们之间转换的技巧、方法或算法。
 - $\text{NFA} \rightarrow \text{DFA}$
 - $\text{DFA} \rightarrow \text{最简DFA}$
 - 非形式描述的语言 $\leftrightarrow$ 正规式
 - 正规式 $\leftrightarrow$ NFA



1. 解答:

(1) $\checkmark$ (2) $\checkmark$ (3) $\times$ (4) $\times$ (5) $\checkmark$

2. [分析]

有限自动机分为确定有限自动机和非确定有限自动机。确定有限自动机的确定性表现在映射 $\delta: Q \times V_T \rightarrow q$ 是单值函数,也就是说,对任何状态 $q \in Q$ 和输入字符串 $a \in V_T$, $\delta(q, a)$ 唯一确定下一个状态。显然,本题给出的是一个确定的有限自动机,它的状态转换图是C中的②。

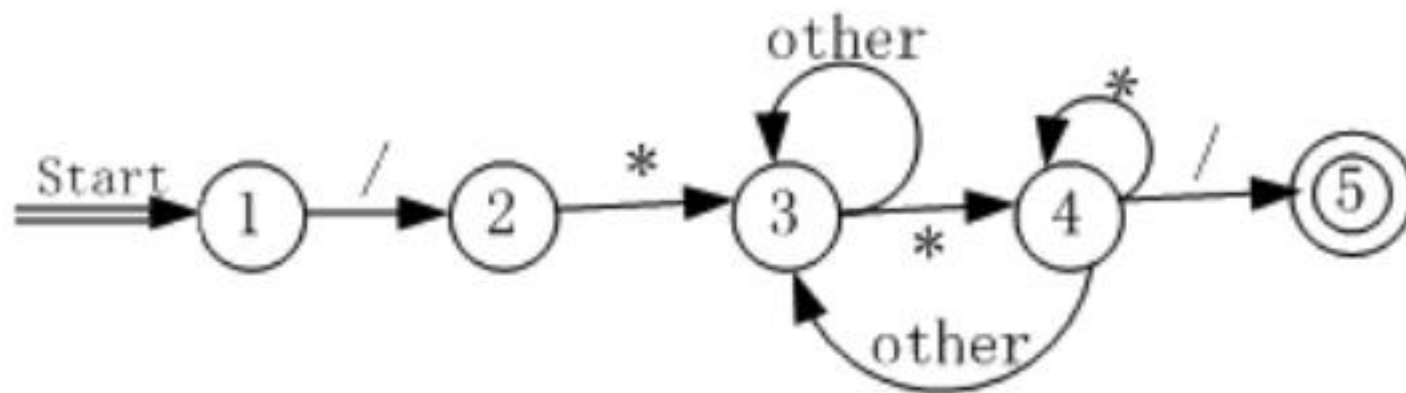
它所接受的语言可以用正则表达式表示为 $00(0|1)^*$,表示的含义为由两个0开始的后跟任意个(包含0个)0或1组成的符号串的集合。

2. 解答: A: ③ B: ② C: ② D: ④



3. 解答：略！

4. 解答：



6. 解答:

(1) $(0|1)^*01$

(2) $((1|2|\dots|9)(0|1|2|\dots|9)^*|\epsilon)(0|5)$

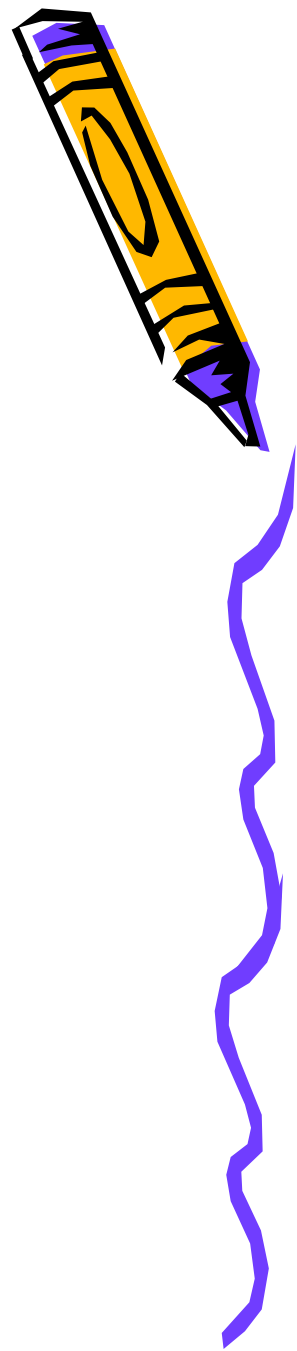
(3) $(0|1)^*(011)(0|1)^*$

(4) $1^*|1^*0(0|10)^*(1|\epsilon)$

(5) $a^*b^*c^*\dots z^*$

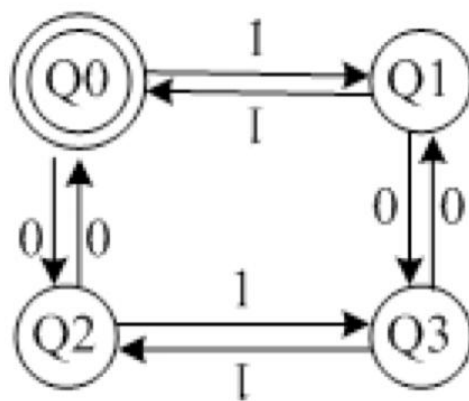
(6) $(0|10^*1)^*10^*$

(7) $(00|11)^*((01|10)(00|11)^*(01|10)(00|11)^*)^*$



(7) 解答:

	0	1
$Q0^*$	Q2	Q1
Q1	Q3	Q0
Q2	Q0	Q3
Q3	Q1	Q2





(8) [分析]

设 S 是符合要求的串， $|S|=2k+1$ ($k \geq 0$)。

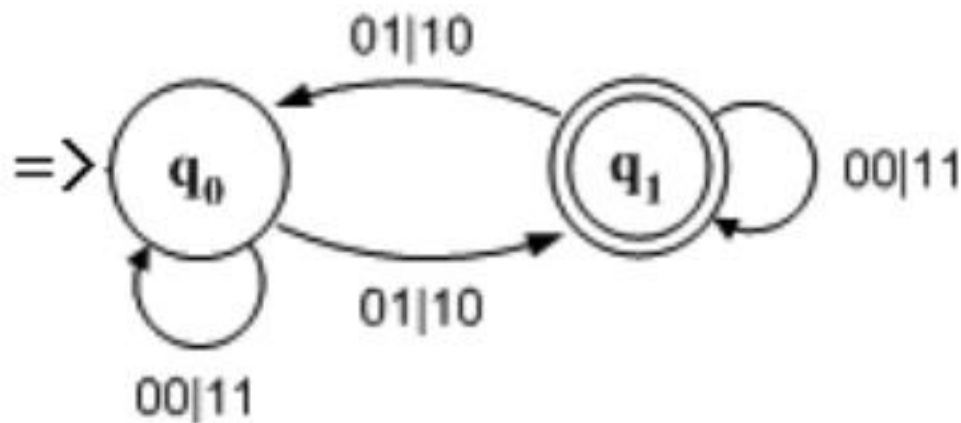
则 $S \rightarrow S_1 0 | S_2 1$ ， $|S_1|=2k$ ($k > 0$)， $|S_2|=2k$ ($k \geq 0$)。

且 S_1 是 $\{0,1\}$ 上的串，含有奇数个0和奇数个1。

S_2 是 $\{0,1\}$ 上的串，含有偶数个0和偶数个1。

考虑有一个自动机 M_1 接受 S_1 ，那么自动机 M_1 如下：
和 $L(M_1)$ 等价的正规式，即 S_1 为：

$((00|11)^*(01|10))^*(01|10)(00|11)^*$



类似的考虑有一个自动机 M_2 接受 S_2 ，那么自动机 M_2 如下：

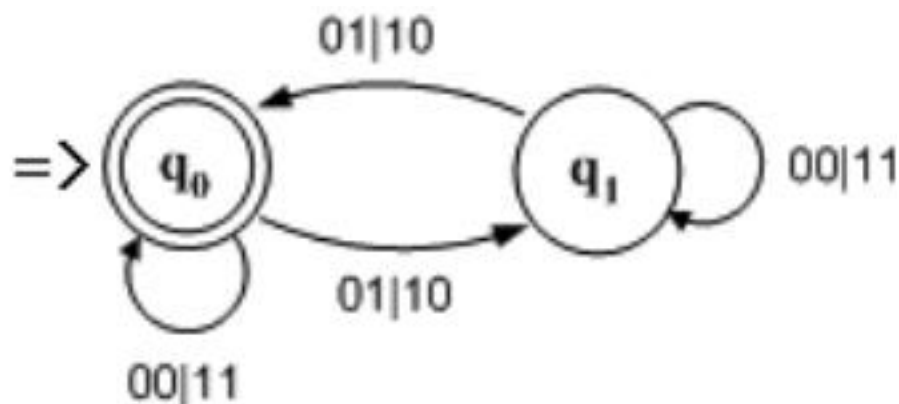
和 $L(M_2)$ 等价的正规式，即 S_2 为：

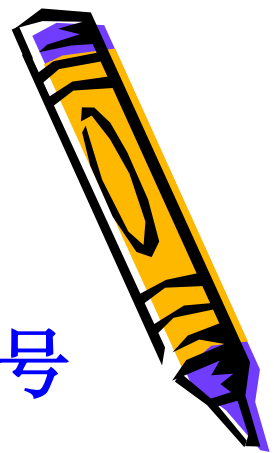
$((00|11)|(01|10)(00|11)^*(01|10))^*$

因此， S 为：

$((00|11)|(01|10)(00|11)^*(01|10))^*(01|10)(00|11)^*0|$

$((00|11)|(01|10)(00|11)^*(01|10))^*1$





7. 解答:

(1) 以0开头并且以0结尾的, 由0和1组成的符号串。

(2) $\{a | a \in \{0,1\}^*\}$

(3) 由0和1组成的符号串, 且从右边开始数第3位为0。

(4) 含3个1的由0和1组成的符号串。

$\{a | a \in \{0,1\}^+, \text{ 且 } a \text{ 中含有 } 3 \text{ 个 } 1\}$

(5) 包含偶数个0和1的二进制串, 即 $\{a | a \in \{0,1\}^*, \text{ 且 } a \text{ 中有偶数个 } 0 \text{ 和 } 1\}$



9. 解答:

(1) DFA $M = (\{0, 1\}, \{q_0, q_1, q_2\}, q_0, \{q_2\}, \delta)$

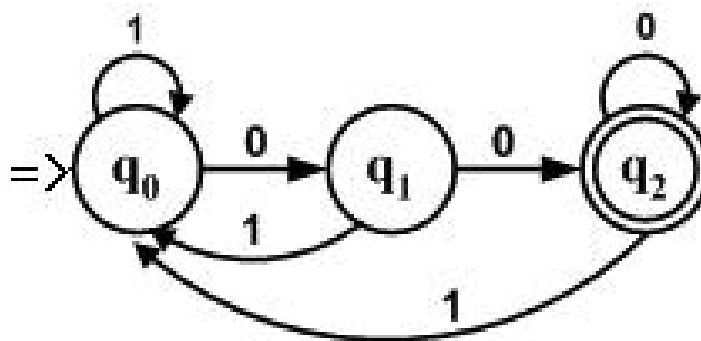
其中 δ 定义如下:

$\delta(q_0, 0) = q_1$ $\delta(q_0, 1) = q_0$

$\delta(q_1, 0) = q_2$ $\delta(q_1, 1) = q_0$

$\delta(q_2, 0) = q_2$ $\delta(q_2, 1) = q_0$

状态转换图为:



(2) 正规式: $1^*01^*01^*01^*$

DFA $M=(\{0, 1\}, \{q_0, q_1, q_2, q_3\}, q_0, \{q_3\}, \delta)$, 其中 δ 定义如下:

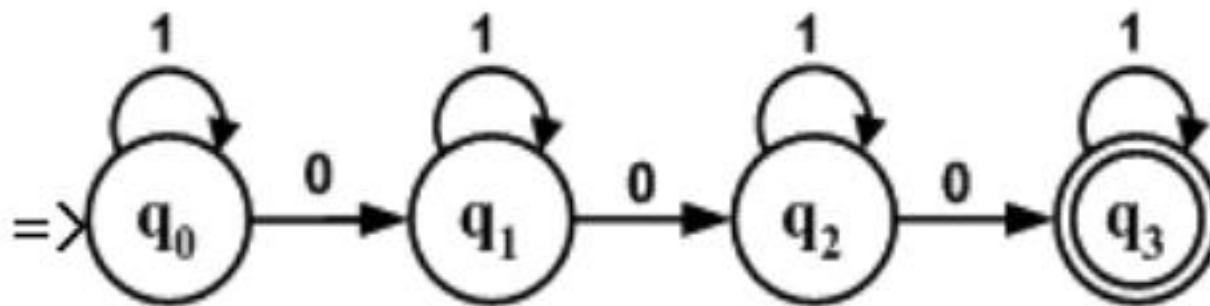
$\delta(q_0, 0)=q_1$ $\delta(q_0, 1)=q_0$

$\delta(q_1, 0)=q_2$ $\delta(q_1, 1)=q_1$

$\delta(q_2, 0)=q_3$ $\delta(q_2, 1)=q_2$

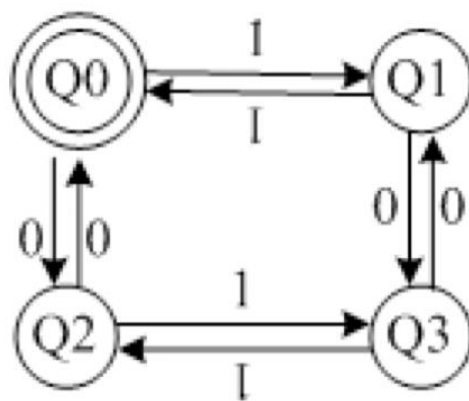
$\delta(q_3, 1)=q_3$

状态转换图为:



(3) 解答:

	0	1
$Q0^*$	Q2	Q1
Q1	Q3	Q0
Q2	Q0	Q3
Q3	Q1	Q2





10. 解答:

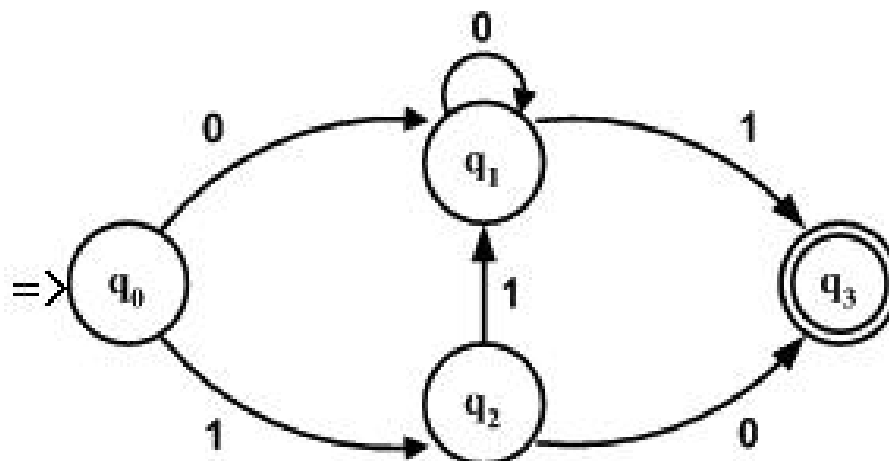
(1) DFA $M = (\{0, 1\}, \{q_0, q_1, q_2, q_3\}, q_0, \{q_3\}, \delta)$, 其中 δ 定义如下:

$\delta(q_0, 0) = q_1$ $\delta(q_0, 1) = q_2$

$\delta(q_1, 0) = q_1$ $\delta(q_1, 1) = q_3$

$\delta(q_2, 0) = q_3$ $\delta(q_2, 1) = q_1$

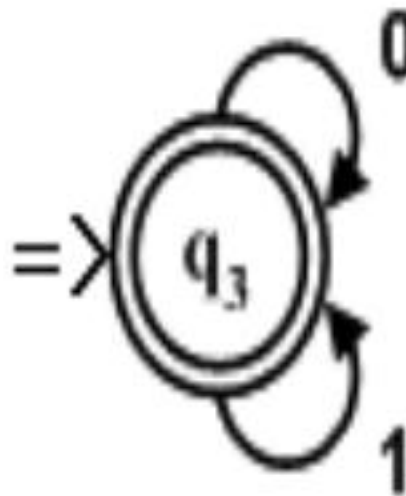
状态转换图为:



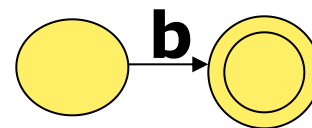
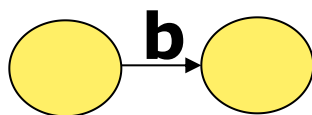
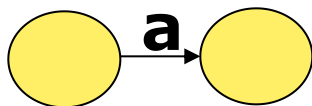
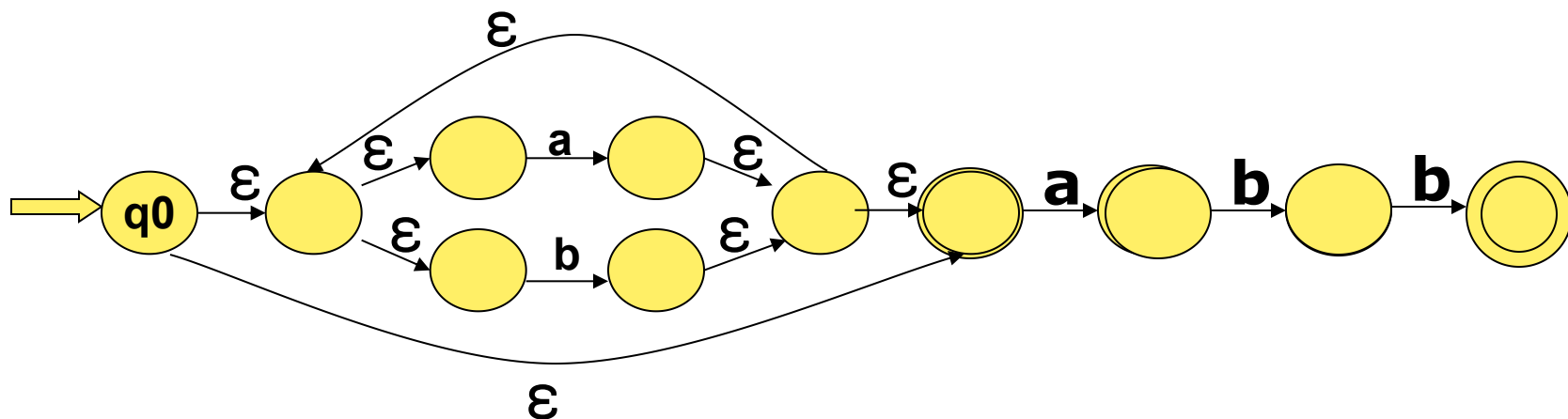
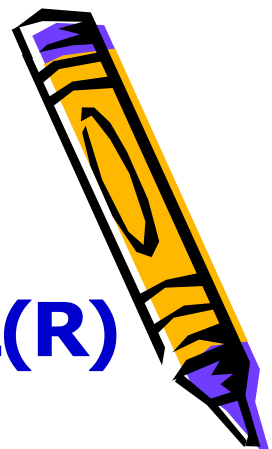
(2) DFA $M=(\{0, 1\}, \{q_3\}, q_3, \{q_3\}, \delta)$, 其中 δ 定义如下:

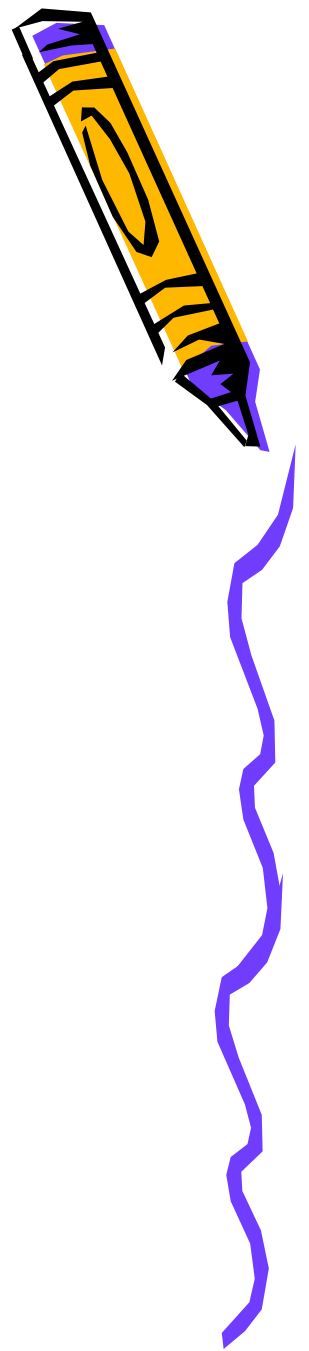
$\delta(q_3, 0)=q_3$ $\delta(q_3, 1)=q_3$

状态转换图为:



正规式 $L(R) = (a|b)^*abb$, 构造NFA使 $L(N)=L(R)$





10 解答:

(3) $(a|b)^*a(a|b)$

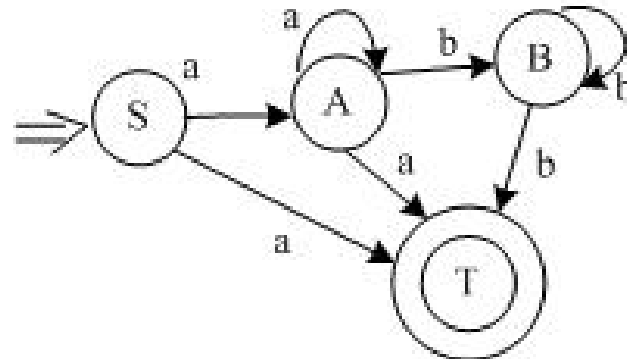
① 求出NFA M:

② 确定化, 得到DFA M:

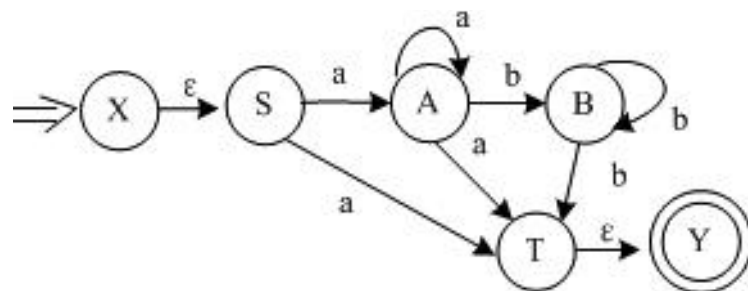
③ 化简: 在第②步中求出的DFA M中没有等价状态, 因此它就是最小化的DFA M。



(4) 解答:
对应的NFA为:



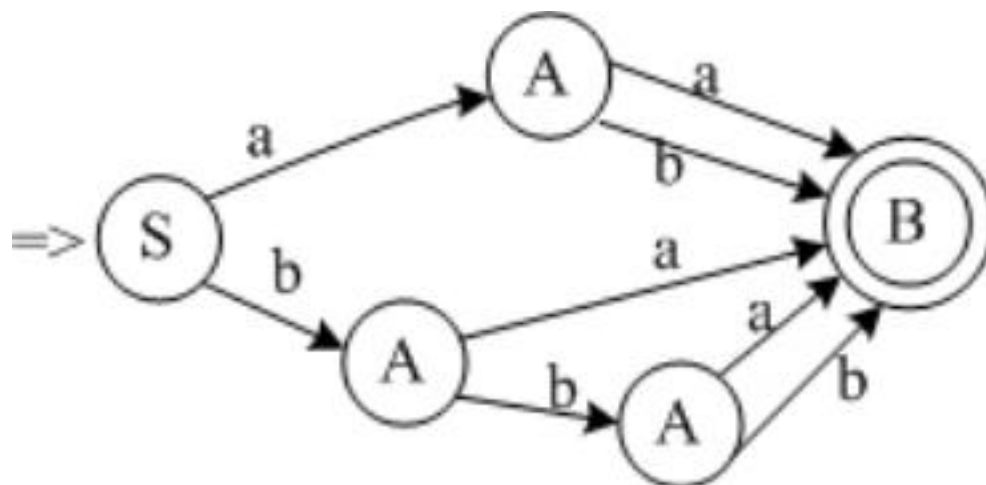
增加状态X、Y，再确定化:



得到的DFA为:



11 (1) . 解答:



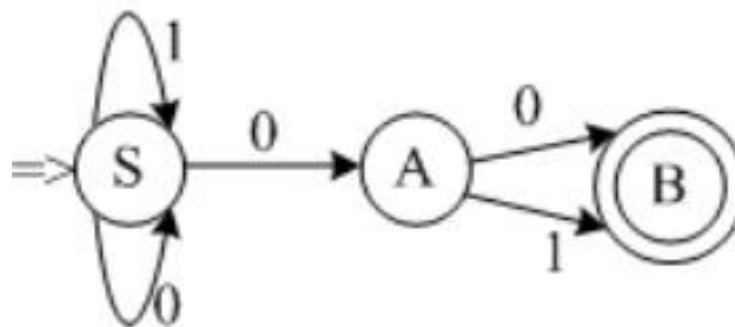
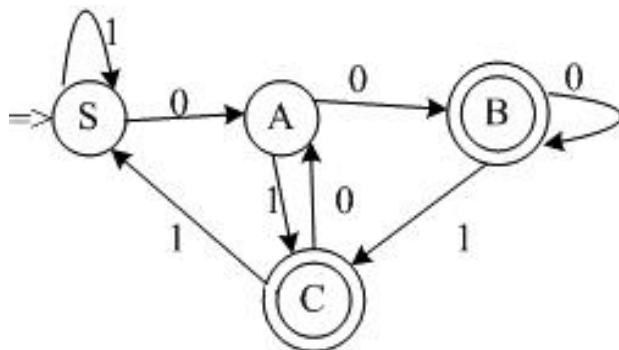


11 (2) . 解答:

正规式为: $(0|1)^*(00|01)$ 化简: $(0|1)^*0(0|1)$

不确定的有穷自动机为:

确定化, 并最小化得到:

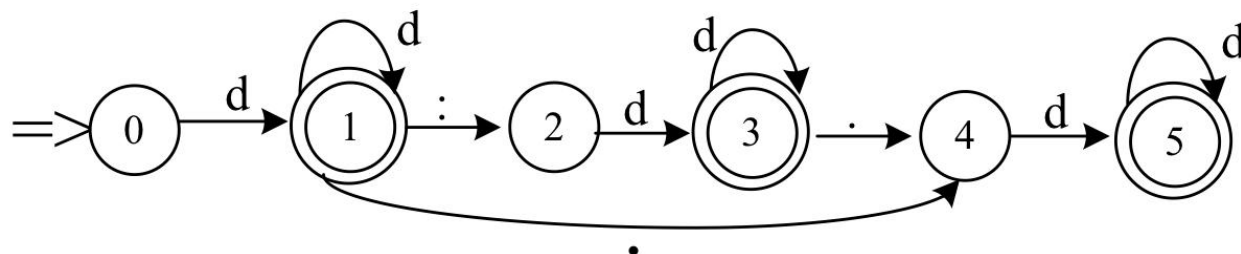
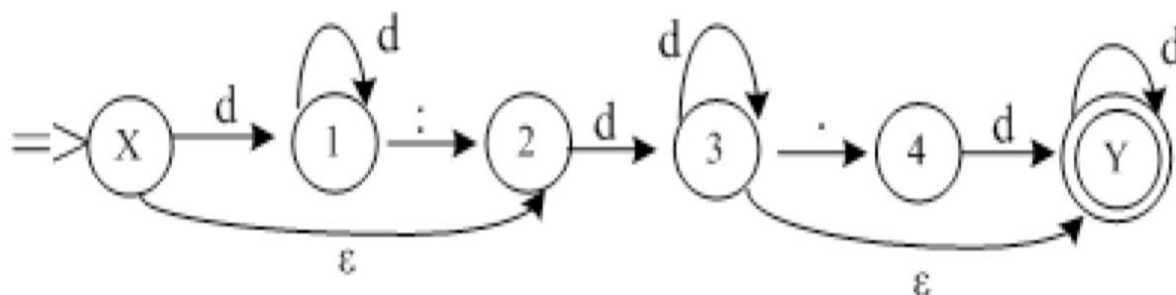


11 (3) .解答:

① 正规式: $(dd^*|\epsilon)dd^*(.dd^*|\epsilon)$, d 代表 $a\sim z$ 的字母

② NFA 为:

③ DFA 为:





11 (4). 解答:
此时编译器认为
/* then part

return q

else

/* else part */

是程序的注释，因此它不可能再发现else 前面的语法错误。

分析 这是注释用配对括号表示时的一个问题。注释是在词法分析时忽略的，而词法分析器对程序采取非常局部的观点。当进入第一个注释后，词法分析器忽略输入符号，一直到出现注释的右括号为止，由于第一个注释缺少右括号，所以词法分析器在读到第二个注释的右括号时，才认为第一个注释处理结束。



词法分析器对源程序采取非常局部的观点，因此象C语言的语句

`fi (a == f(x)) ...`

中，词法分析器把fi当作一个普通的标识符交给编译的后续阶段，而不会把它看成是关键字if的拼写错。

PASCAL语言要求作为实型常量的小数点后面必须有数字，如果程序中出现小数点后面没有数字情况，它由词法分析器报错。

